ChatGPT

Executive Summary

The project is at an apparent *quiescent* state: the surfacer reports `active_work=0, next_automatic_action=NONE`, with multiple surfaced items but the top item marked `action_required=false` and `disposition=UNSEEN`. This raises the crucial question: **is the system truly done, or is something hidden?** We must **prompt ourselves** to check whether any remaining unseen obligation could alter the authorized consequence. This involves a dual analysis: *compression* (which distinctions in the recovered state can be erased) and *acquisition* (which observations are still relevant). We design a Self-Prompt–QUIESCENCE to probe this state and propose concrete tests, data structures, and policies to **justify or falsify quiescence**. The key insight is that a system can safely stop only when *no admissible distinction can change the lawful dispatch*. This principle is mirrored in decision-theoretic *value-of-information* (no value if decision unchanged【24†L262-L270】) and in formal sensor-selection: pick the smallest observation set sufficient for the control property【29†L25-L33】. Our plan is to generate a structured prompt that the live system can execute, and then *stop exactly when no further discrimination matters*.

Key results:

- A concrete **SELF-PROMPT—QUIESCENCE** is formulated (below) that uses the system's own state to generate the next research question.
- An **execution checklist** and minimal pseudo-tests are given to (a) verify the stopping condition, (b) build counterexample states (active_work=0 but some action possible), and (c) produce *proof-carrying* DO_NOT_OBSERVE *witnesses* (bound to the actual candidates and gate version) when observations are pruned.
- **Data structures** (`ObservationSelectionWitness`, certificate, `CandidateSetID`, `GateVersion`) and test APIs are sketched so that each DO_NOT_OBSERVE decision carries a verifiable record, akin to W3C PROV or in-toto attestations【14†L50-L58】【17†L198-L207】.
- A proposed *prompt lineage* P0…P3 shows how successive self-prompts refine the task, and defines a stopping criterion **QUIESCENCE_EARNED**.
- We list prioritized **implementation tasks** and negative tests (e.g. disallow laundering UNKNOWN→NONINTERFERING) with simple one-line acceptance checks.
- Tables summarize required witness fields and their purpose, and task priorities/effort.
- Finally, a **mermaid flowchart** outlines the quiescence-assay logic, and a sample JSON witness + test vectors illustrate the certificates.

Throughout, we ground our design in related work: *provenance standards* (W3C PROV) and *attestation frameworks* (in-toto) emphasize recording evidence【14†L50-L58】【17†L198-L207】, while *minimal-information transition systems* and *sensor-selection* literature justify our objective of "no more (and no less) information than needed"【38†L29-L34】【29†L25-L33】.

1. SELF-PROMPT — QUIESCENCE

STATE: The system reports `active_work=0` and `next_automatic_action=NONE`. Multiple tasks remain surfaced; the top item shows `action_required=false` and `disposition=UNSEEN`. The surfacer's summary: **"Nothing is queued, nothing needs action, but items are visible."**

TENSION: This situation *looks like* no work is pending, but is that truly so? Some surfaced objects are still **UNSEEN** or **UNRESOLVED**. Are they *legitimately idle* or is work hidden behind them? In other words, is visible quiescence genuine, or an illusion masking unfinished obligations?

QUESTION: *What distinguishes genuine quiescence from overlooked work?* In this state, what evidence would prove that continuing (or stopping) is correct? Concretely: "Can any unresolved distinction in the current state still change the lawful dispatch?"

PRESSURE: We suspect that *visibility* (item is shown) ≠ *obligation*. Maybe an item's `disposition=UNSEEN` means we haven't fully processed it. Maybe some checks are incomplete. Or maybe everything truly is finished. We must pressure-test the assumption of quiescence: could there be a **counterfactual** in which a different outcome of an "unseen" fact *would alter* the next consequence?

SURPRISE: A shock would be to find a surfaced item labeled "UNSEEN, action_required=false" that *still* holds a phantom unresolved issue. For example, if choosing one more observation or filter would flip the decision from ADMIT to WITHHOLD. Discovering such a case would overturn the "NONE" action verdict.

ASSAY: Ask the system: "Given the current state, identify *any* admissible observation or distinction whose absence could change the authorized consequence. Prove either (a) that no such distinction exists, or (b) present one counterexample case." In practice, we prompt the system to recompute as if each possible bit of hidden information were resolved differently, and check if the dispatch changes.

Concrete Prompt:

```
STATE: Surfacer reports active_work=0, next_action=NONE. Top item has
disposition=UNSEEN and action_required=false, but other surfaced items exist.

TENSION: This looks like everything is done, yet some items are UNSEEN or
UNRESOLVED. Have we missed anything?

QUESTION: Can any unresolved distinction (observation or state bit) still alter
the lawful dispatch outcome? Is the system truly quiescent?

PRESSURE: Challenge the assumption of quiescence. Could a remaining unseen
candidate or excluded reality change the decision? Which one?

SURPRISE: Exhibit an example where an omitted distinction flips the decision, or
prove none exists.
```

```
ASSAY: Search the current state for hidden work. For each surfaced item and
observation, simulate its possible values. If all admissible values yield the
same lawful successor, then quiescence is justified; otherwise, continue.
```

We then **execute** this self-prompt against the live repository state. The system should attempt to answer it and possibly surface a new question or confirm quiescence.

2. Quiescence Verification: Checklist & Test Plan

To *verify* quiescence and to try to falsify it, we propose the following checklist and minimal pseudocode tests:

- **a. Gather Current State:** Query the surfacer data: all surfaced items, their dispositions, pending consequences, and the current authorized consequence (if any). Verify `active_work=0` and `next_action=NONE`.
- **b. Enumerate Remaining Observations:** For each surfaced item or "excluded reality", list all *admissible observations* (sensor readings, world splits, consequences of derivation rules) that are currently *unknown or unresolved*.
- **c. For each observation `O`:**
- List its *admissible outcomes* {o₁, ..., o_k}.
- **Simulate** the system's dispatch if we set `O=o_i` (leaving the rest of state as is). That is, compute *conditional dispatch*:

```
Q_i = GateConsequenceQuotient(E | O=o_i)  → dispatch D_i
```

- Compare each D_i to the *base dispatch* D₀ (with O unresolved).

- **d. Check Dispatch Variance:**

- If **any** pair D_i ≠ D_j or D_i ≠ D_0, then some observation matters: quiescence is *not* justified (active_work should > 0). **Report a counterexample:** e.g., "If O=val1 then dispatch=ADMIT, if O=val2 then dispatch=WITHHOLD."

- If *all* D_i are identical to D₀ for every tested O, then no single-observation change affects the decision.

- **e. Joint Observations:** Also test minimal observation *sets*: some distinctions are only discriminating in combination. For two unresolved observations O1, O2, simulate all outcome pairs (o1, o2) and check if any joint assignment changes dispatch. (If individually non-discriminating but jointly discriminating, quiescence is false.)

- **f. Exhaustivity Check:** Ensure the tested observation set covers all admissible options. If some outcomes are excluded/unknown in tests, flag it as "UNKNOWN – continue work" rather than "noninterfering."

- **g. Produce Certificates:** Whenever the analysis yields **DO_NOT_OBSERVE** for an observation (meaning it cannot alter dispatch), output a *certificate (witness)* recording all outcomes and dispatch results (see next section). Likewise, if we conclude quiescence (no candidate can alter), output a DO_NOT_OBSERVE certificate for the entire candidate set.

- **h. Counterexample Construction:** If the check finds an observation or set that *does* alter the decision, keep work active. A minimal counterexample state can be constructed by modifying the "unseen" part of the state: e.g., take one hidden predicate `P`, set it to a value that flips admission, and treat that as a new needed action.

Minimal Pseudocode (outline):

```
def check_quiescence(state, gate, policy):
    candidates = enumerate_unseen_observations(state)
    base_dispatch = gate.apply(state, policy)
    evidence = []
    for obs in candidates:
        outcomes = admissible_outcomes(obs)
        cond_dispatches = []
        for val in outcomes:
            new_state = state.with_observation(obs, val)
            d = gate.apply(new_state, policy)
            cond_dispatches.append(d)
        if any(d != base_dispatch for d in cond_dispatches):
            return False, f"{obs} is discriminating", cond_dispatches
        evidence.append((obs, outcomes, base_dispatch))
    # Optionally test joint obs sets...
    return True, "No single observable alters dispatch", evidence
```

Test vectors: For example, suppose the top surfaced item had two unresolved branches *a* and *b*. We create a dummy state where `O` = "which branch?" with values {a,b}. If gate yields ADMIT for a and WITHHOLD for b, the function returns False with that fact. If gate yields ADMIT for both, returns True.

3. Witness Data Structures & APIs

To *prove* any DO_NOT_OBSERVE or UNKNOWN claim, we define data structures that carry the evidence:

- `ObservationSelectionWitness` (struct or JSON): a detailed record for each DO_NOT_OBSERVE decision. Fields include:
- `consequenceType`: the target consequence being examined (e.g. "delegateOperation(C,B,P)").
- `basisID`, `policyID`: identifiers (or hashes) of the relevant basis and policy versions (to bind scope).
- `candidateSetID`: a hash (e.g. SHA256) of the sorted definitions of all evaluated observations {O1,...,On} (their code paths or unique IDs). This ensures the witness is tied to exactly these candidates.

- `candidate_count`: number of observations evaluated.
- `baseQuotientID`, `baseDispatch`: identity of the base quotient (hash of the equivalence classes) and the resulting lawful dispatch before observing.
- `per_candidate`: a list of entries, one per candidate observation, each with:
    - `observationID` (unique key),
    - `admissible_outcomes` (list of values tested),
    - `excluded_outcomes` (if any outcomes were not tested),
    - `conditionalDispatches`: mapping from outcome value → dispatch (ADMIT/WITHHOLD),
    - `relevanceStatus`: {"DISCRIMINATING", "NONINTERFERING", "UNKNOWN"},
    - `cost` (if a cost vector is used).
- `selected_candidate` (if a question was chosen; else null),
- `selection_rule` (e.g. "lowest_cost", "unused" if no selection was made),
- `selection_status`: {"NO_CANDIDATES", "ALL_NONINTERFERING", "SELECTED"},

- `failed_obligations`: (optional) list of unresolved distinctions if any.

- `ObservationSelectionCertificate`: this is a signed or hashed object that includes the witness plus binding fields:

- `version`: identifier for the gate function (e.g. GateConsequenceQuotient v1.2),
- `timestamp` or computation step,
- `witness` (the struct above),

- `certificate_hash`: a hash of all contents for tamper-proofing.

- **Hashing Scheme:**

- `CandidateSetID = H( canonicalize(obs_def1,…,obs_defN) )`, where each `obs_def` includes the source code identity of the observation (e.g. name, allowed range). This prevents name collisions.
- `GateVersion` binding: we include e.g. `gateVersion = H(code_of_GateConsequenceQuotient)` or a release tag, so a change in gate semantics invalidates old witnesses.

- Similarly, `basisID = H(code_of_reconstruction)` and `policyID = H(code_of_policy)`.

- **API Suggestions:**

- `GenerateWitness(candidateObs, baseDispatch, results, costVector, gateID)`: produces and returns the `ObservationSelectionCertificate`.
- `VerifyWitness(cert, gate, state, policy)`: checks all listed conditionalDispatches against fresh gate evaluations, ensures hashes match, and confirms no undisclosed candidate could alter dispatch. Returns true/false.

These structures ensure that **every DO_NOT_OBSERVE or quiescence claim carries a "proof"**. This aligns with W3C PROV's emphasis on verifiable derivation chains【14†L50-L58】 and with in-toto's use of

attestations to prove that “no step was tampered with”【17†L198-L207】. In our case, the attestation is: “Even if we asked any of these questions, the answer wouldn’t change the decision.”

4. Self-Prompt Lineage (P0–P3)

We track how our prompting evolves. Each entry is a self-generated question derived from the live state, along with its effect:

- **P0:** “SELF-PROMPT–QUIESCENCE (initial)”.
- *Generated by State:* active_work=0, next_action=NONE, top item UNSEEN.
- *Why Interesting:* Tests if *any* unseen distinction can change the consequence (i.e. if quiescence is false).
- *Caused:* The system ran the quiescence assay above.
- *Reality Taught:* If no change was found, we gain confidence quiescence might hold. If a change was found, we resume work (details under P2).
- *Expired:* This prompt’s question is resolved when we determine whether any single observation affects the result. (If none do, P0’s task is done.)

- **P1:** “Check jointly noninterfering observations.”

- *State:* P0 found all individual observations were “NONINTERFERING”, but multiple remained.
- *Why Interesting:* Even if each question alone doesn’t change the decision, *together* they might (as in XOR cases).
- *Caused:* The system tests pairs (or small sets) of observations for joint discrimination.
- *Reality Taught:* Either (a) found a joint distinguishing pair (so quiescence false, see P2), or (b) none exist (strengthening quiescence).
- *Expired:* Once joint-tests confirm no pair changes dispatch (or one pair does), the question is answered.

- **P2:** “Is the witness binding correct and complete?”

- *State:* We intend to output DO_NOT_OBSERVE.
- *Why Interesting:* Ensures the certificate is properly scoped to this candidate set and gate version.
- *Caused:* The system formats an `ObservationSelectionWitness`, including hash of candidate definitions and gate code.
- *Reality Taught:* Verifier code now can check the certificate against the exact candidates considered and the gate implementation.
- *Expired:* This prompt is done when a valid, denominator-bound witness is generated.

- **P3 (final):** “QUIESCENCE_EARNED – Stop.”

- *State:* No observation (single or joint) can alter the decision, and a valid witness is recorded.
- *Why Interesting:* We should only stop if **really** no further work can affect consequences.
- *Caused:* The system halts further work and declares quiescence.

- *Reality Taught:* If truly nothing unresolved is relevant, we have earned quiescence.
- *Expired:* Project ends unless new inputs arrive (the lineage stops here).

PROMPTS THAT FAILED PRODUCTIVELY: e.g. a prompt seeking "find a missing constraint" might have turned up an undiscovered obligation. (Any prompt whose answer indicated more work needed.)

PROMPTS MOST INFORMATION: P0 and P1, since they map out the space of observations and exposed hidden dependencies.

PROMPTS GENERATED MOST ACTIVITY BUT LITTLE INFO: A prompt probing already-resolved items (if any) that led only to `NO_CANDIDATES` states.

NEW QUESTIONS EMERGED: "What is the correct scope of the witness?" (leading to P2), "How to detect joint observations?" (leading to P1).

MODEL CHANGES: The system now distinguishes *"unknown ≠ noninterfering"* and requires a typed witness.

ROOT CAUSES: We discovered that previous logic had status-donation flaws (UNKNOWN being treated as NONINTERFERING), so needed domain-specific handling.

GENERALIZATIONS KILLED/SURVIVED: Survived: "should preserve only distinctions that can affect decisions." Killed: "any unknown means safe to stop."

CROSS-DOMAIN CONNECTIONS: The problem bridged software supply-chain attestation (in-toto) and decision-theory (value-of-information), blending them in a runtime context.

CURRENT SELF-PROMPT: If quiescence was not earned, continue with the pending work. Otherwise, **stop**.

TARGET: Ultimately, *no new prompt* is needed when the system proves "no further distinction matters."

5. Prioritized Tasks, Negative Tests, and Criteria

Task	Effort	Risk	Description & Acceptance
1. Implement **ObservationSelectionWitness**	High	Medium	Define struct/JSON as above. Acceptance: witnesses contain all fields (IDs, outcomes, dispatches, etc.) with correct scopes.
2. Tie **CandidateSetID & GateVersion**	Medium	Low	Compute hashes of canonical obs set and gate code. Acceptance: modifying any candidate or gate changes witness hash.
3. Disentangle **UNKNOWN vs NONINTERFERING**	Medium	Low	Treat UNKNOWN candidates separately (e.g. status "UNRESOLVED"). Acceptance: UNKNOWN never auto-turns NONINTERFERING.

Task	Effort	Risk	Description & Acceptance
4. Generate **Per-candidate Dispatch Table**	High	Medium	For each question, record outcome→dispatch. Acceptance: every witnessed DO_NOT_OBSERVE includes a full table.
5. Joint-Observation Testing	High	High	Check combinations of 2+ observations. Acceptance: if any pair jointly discriminates, quiescence check fails appropriately.
6. Cost-based **Selector (simple)**	Low	Low	Compute cost per question. Acceptance: still picks cheapest of NONINTERFERING questions (but not used until needed).
7. **Dominance Filtering** of observations	Medium	Medium	Prune questions if another cheaper one refines same partition. Acceptance: no dominated obs left on frontier.
8. Boundary **Do_Not_Observe Certificates**	Medium	Medium	Issue signed or hashed certificates. Acceptance: any DO_NOT_OBSERVE includes witness that verifies.
9. **Admissibility of Observation** (A_O)	High	High	Compute if asking O is allowed (policy/time). Acceptance: inadmissible O never selected, even if informative.
10. **Integration Testing**	High	Low	End-to-end tests of quiescence. Acceptance: in quiescent state, system prints `DO_NOT_OBSERVE` with correct witness for all candidates; in non-quiescent state, it continues.

- **Negative test (UNKNOWN laundering):** If `EvaluateObservationRelevance` returns `UNKNOWN` for an observation, the selector **must not** label it NONINTERFERING and stop. *Test:* Introduce a candidate with incomplete coverage (simulate excluded outcome); check that the final decision is either “RELEVANCE_UNRESOLVED” or similar, not `DO_NOT_OBSERVE`.
- **Negative test (over-discard):** Two unresolved worlds mapping to ADMIT/ADMIT, observation distinguishes them: ensure DISCRIMINATING status *not* lead to DO_NOT_OBSERVE. *Acceptance:* the obs is marked irrelevant only if baseDispatch is equal under both.
- **Positive test (witness completeness):** For a trivial state (single unresolved 0/1 question, no effect), check the system emits an `ALL_NONINTERFERING` witness. *Acceptance:* witness shows both outcomes leading to ADMIT (say) and selection_status=ALL_NONINTERFERING.
- **Edge test (joint synergy):** Create two binary questions where each alone is noninterfering but together would change dispatch (XOR). The system should NOT stop after finding each alone noninterfering. *Acceptance:* the joint test detects change, preventing quiescence.
- **Stop criterion test:** After all tests, if nothing found, the system prints a final certificate for DO_NOT_OBSERVE(all) and halts. *Acceptance:* `next_automatic_action` remains NONE and surfacer enters QUIESCENCE state.

6. Tables

(a) Minimal Witness Fields vs. Purpose

Field	Purpose
consequenceType	Identifies which specific consequence (C,B,P) the witness refers to (tight scope).
basisID, policyID	Bind witness to the specific code/version of the state-reconstruction and policy.
candidateSetID	Hash of canonicalized observation definitions; ensures witness covers exactly those questions.
candidate_count	Sanity check on number of candidates evaluated.
baseQuotientID	Identity of the consequence-equivalence partition for the current state.
baseDispatch	The lawful successor (ADMIT/WITHHOLD) before asking any new question.
per_candidate.*	For each obs: record ID, admissible vs excluded outcomes, and dispatches under each outcome.
relevanceStatus	(per candidate) DISCRIMINATING / NONINTERFERING / UNKNOWN (precision of conclusion).
cost	(per candidate) optional cost metric for choosing questions.
selected_candidate	Which observation was chosen (if any).
selection_rule	Reason for choice (e.g. “lowest_cost”).
selection_status	NONE/CANDIDATE_CHOSEN/ALL_NONINTERFERING/NO_CANDIDATES.
failed_obligations	If quiescence false, list distinctions that break the decision (for debugging).

(b) Prioritized Tasks

Task	Effort	Risk
Build `ObservationSelectionWitness` type and JSON output	High	Medium (complex data)
Incorporate hashed `CandidateSetID` and gate version in witness	Medium	Low (straightforward hashing)
Disambiguate `UNKNOWN` status in selector logic	Medium	Low (simple check logic)
Emit full per-candidate dispatch tables in witness output	High	Medium (ensures correctness)
Joint-observation relevance testing	High	High (combinatorial complexity)

Task	Effort	Risk
Cost-based question ranking	Low	Low (greedy heuristic)
Dominance and minimal-set solver	Medium	Medium (NP-hard aspects)
Generate and sign `DO_NOT_OBSERVE` certificates	Medium	Medium (needs stable hashing)
Admissibility check for acquiring observations (`A_O`)	High	High (policy integration)
End-to-end quiescence validation tests	Medium	Low (test scenario design)

7. Quiescence Assay Flowchart and Witness Example

```
flowchart TD
    A[Start] --> B{active_work > 0?}
    B -- Yes --> Z[Continue work]
    B -- No --> C{next_action == NONE?}
    C -- No --> Z
    C -- Yes --> D{Any surfaced items?}
    D -- No --> E[Quiescent: Done (no items left)]
    D -- Yes --> F{Top item action_required?}
    F -- Yes --> Z
    F -- No --> G[Enumerate all unresolved distinctions]
    G --> H[Test each distinction]
    H --> I{Does any change dispatch?}
    I -- Yes --> Z[Reset work = non-quiescent (return to normal loop)]
    I -- No --> J{Combine distinctions?}
    J -- Yes --> H (loop over combinations)
    J -- No --> K[Emit DO_NOT_OBSERVE certificate & stop]
    K --> L[Project is quiescent – no further action]
```

Sample `ObservationSelectionWitness` (JSON):

```
{
  "consequenceType": "delegateOp(C1,BX,PV)",
  "basisID": "hash_of_basis_code",
  "policyID": "hash_of_policy_code",
  "candidateSetID": "sha256(obs1_def || obs2_def)",
  "candidate_count": 2,
  "baseQuotientID": "hash_of_Q(C1,BX,PV)",
  "baseDispatch": "ADMIT",
  "per_candidate": [
```

```
    {
      "observationID": "Obs1",
      "admissible_outcomes": ["a","b"],
      "excluded_outcomes": [],
      "conditionalDispatches": {"a": "ADMIT", "b": "ADMIT"},
      "relevanceStatus": "NONINTERFERING",
      "cost": 10
    },
    {
      "observationID": "Obs2",
      "admissible_outcomes": ["x","y","z"],
      "excluded_outcomes": [],
      "conditionalDispatches": {"x":"ADMIT","y":"ADMIT","z":"ADMIT"},
      "relevanceStatus": "NONINTERFERING",
      "cost": 5
    }
  ],
  "selected_candidate": null,
  "selection_rule": "NONE",
  "selection_status": "ALL_NONINTERFERING",
  "failed_obligations": []
}
```


Test Vector Example:

- *State:* One candidate observation “IsUserOver18” with outcomes {true, false}.
- *Gate Behavior:* If true→ADMIT, if false→ADMIT (no effect).
- *Expected:* The system recognizes no change. It emits a witness with both outcomes mapping to ADMIT, `relevanceStatus=NONINTERFERING`, and final `ALL_NONINTERFERING`.

Another vector:

- *State:* Two candidates “hasCredential” and “validChecksum”. Alone neither affects dispatch, but if both false → WITHHOLD.
- *Expected:* The system should **not** stop after single tests, should flag joint discrimination.

Sources

We anchor our approach in established theory and practice. The W3C PROV model defines provenance as “information about entities, activities, and people involved in producing data” useful for trust【14†L50-L58】. Similarly, in-toto’s attestation framework issues verifiable claims about software production steps【17†L198-L207】; our witnesses serve the same role for runtime decisions. The pursuit of *minimal sufficient information* in policies is well-studied: Sakcak et al. prove that minimal-information transition systems exist and yield unique minimal representations for decision tasks【38†L29-L34】. Likewise, selecting the minimal sensor/observation set that still achieves a formal property (e.g. observability) is NP-hard but well-defined【29†L25-L33】. Finally, decision theory teaches that information has value only if it can change the

decision: the “expected value of perfect information” (EVPI) is zero when the best choice remains the same【24†L262-L270】. We leverage these insights to make the machine *only* acquire or retain distinctions that can lawfully affect its successor state.
